



Python Programming - 2301CS404

Lab - 15 (1)

Roll No.: 418

Name: Bhavik A. Parmar

01) Write a Program to create a class by name Students, and initialize attributes like name, age, and grade while creating an object.

```
In [6]: class Students:
        def __init__(self, name, age, grade):
            self.name = name
            self.age = age
            self.grade = grade

s1 = Students("Bhaviik", 18, "A")

print("Name:", s1.name)
print("Age:", s1.age)
print("Grade:", s1.grade)
# Name: Bhaviik
# Age: 18
# Grade: A

#OR
# class Students:
#     pass

# s1 = Students()
# s1.name = "Bhavik"
# s1.age = 18
# s1.grade = "A"

# print(s1.name, s1.age, s1.grade)
# Bhavik 18 A
```

Name: Bhaviik

Age: 18

Grade: A

02) Create a class named Bank_Account with Account_No, User_Name, Email, Account_Type and Account_Balance data members. Also create a method GetAccountDetails() and DisplayAccountDetails(). Create main method to demonstrate the Bank_Account class.

```
In [8]: class Bank_Account:

    def __init__(self, acc_no, user_name, email, acc_type, balance):
        self.Account_No = acc_no
        self.User_Name = user_name
        self.Email = email
        self.Account_Type = acc_type
        self.Account_Balance = balance

    def GetAccountDetails(self):
        self.Account_No = input("Enter Account Number: ")
        self.User_Name = input("Enter User Name: ")
        self.Email = input("Enter Email: ")
        self.Account_Type = input("Enter Account Type: ")
        self.Account_Balance = float(input("Enter Account Balance: "))

    def DisplayAccountDetails(self):
        print("\n--- Account Details ---")
        print("Account No:", self.Account_No)
        print("User Name:", self.User_Name)
        print("Email:", self.Email)
        print("Account Type:", self.Account_Type)
        print("Account Balance:", self.Account_Balance)

def main():
    acc = Bank_Account("", "", "", "", 0)

    # Calling methods
    acc.GetAccountDetails()
    acc.DisplayAccountDetails()

main()
```

```
--- Account Details ---
Account No: 12345
User Name: Bhavik
Email: er.bhavik5202@gmail.com
Account Type: Saving
Account Balance: 50000.0
```

03) WAP to create Circle class with area and perimeter function to find area and perimeter of circle.

```
In [9]: import math

class Circle:
```

```

def __init__(self, radius):
    self.radius = radius

def area(self):
    return math.pi * self.radius * self.radius

def perimeter(self):
    return 2 * math.pi * self.radius

def main():
    r = float(input("Enter radius: "))

    c = Circle(r)

    print("Area of Circle:", c.area())
    print("Perimeter of Circle:", c.perimeter())

main()

```

Area of Circle: 78.53981633974483

Perimeter of Circle: 31.41592653589793

04) Create a class for employees that includes attributes such as name, age, salary, and methods to update and display employee information.

```

In [11]: class Employee:

    def __init__(self, name, age, salary):
        self.name = name
        self.age = age
        self.salary = salary

    def update_details(self):
        self.name = input("Enter new name: ")
        self.age = int(input("Enter new age: "))
        self.salary = float(input("Enter new salary: "))

    def display_details(self):
        print("\n--- Employee Details ---")
        print("Name:", self.name)
        print("Age:", self.age)
        print("Salary:", self.salary)

def main():
    emp = Employee("Bhavik", 18, 25000)

    print("Initial Details:")
    emp.display_details()

    print("\nUpdate Details:")
    emp.update_details()

    print("\nUpdated Details:")
    emp.display_details()

main()

```

Initial Details:

--- Employee Details ---

Name: Bhavik

Age: 18

Salary: 25000

Update Details:

Updated Details:

--- Employee Details ---

Name: Sunil

Age: 25

Salary: 1000000.0

05) Create a bank account class with methods to deposit, withdraw, and check balance.

```
In [13]: class BankAccount:

    def __init__(self, name, balance=0):
        self.name = name
        self.balance = balance

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print("Amount Deposited:", amount)
        else:
            print("Invalid deposit amount!")

    def withdraw(self, amount):
        if amount > self.balance:
            print("Insufficient balance!")
        elif amount <= 0:
            print("Invalid withdrawal amount!")
        else:
            self.balance -= amount
            print("Amount Withdrawn:", amount)

    def check_balance(self):
        print("Current Balance:", self.balance)

def main():
    acc = BankAccount("Bhavik", 1000)

    while True:
        print("\n1. Deposit")
        print("2. Withdraw")
        print("3. Check Balance")
        print("4. Exit")

        choice = int(input("Enter choice: "))

        if choice == 1:
            amt = float(input("Enter deposit amount: "))
            acc.deposit(amt)
```

```

elif choice == 2:
    amt = float(input("Enter withdrawal amount: "))
    acc.withdraw(amt)

elif choice == 3:
    acc.check_balance()

elif choice == 4:
    print("Thank you!")
    break

else:
    print("Invalid choice!")

main()

```

1. Deposit
 2. Withdraw
 3. Check Balance
 4. Exit
 Current Balance: 1000

1. Deposit
 2. Withdraw
 3. Check Balance
 4. Exit
 Current Balance: 1000

1. Deposit
 2. Withdraw
 3. Check Balance
 4. Exit
 Thank you!

06) Create a class for managing inventory that includes attributes such as item name, price, quantity, and methods to add, remove, and update items.

```

In [14]: class Inventory:

    def __init__(self):
        # Dictionary to store items
        # Format: {item_name: {"price": value, "quantity": value}}
        self.items = {}

    def add_item(self, name, price, quantity):
        if name in self.items:
            print("Item already exists! Updating quantity...")
            self.items[name]["quantity"] += quantity
        else:
            self.items[name] = {"price": price, "quantity": quantity}
            print("Item added successfully!")

    def remove_item(self, name):
        if name in self.items:
            del self.items[name]
            print("Item removed successfully!")
        else:
            print("Item not found!")

```

```

def update_item(self, name, price=None, quantity=None):
    if name in self.items:
        if price is not None:
            self.items[name]["price"] = price
        if quantity is not None:
            self.items[name]["quantity"] = quantity
        print("Item updated successfully!")
    else:
        print("Item not found!")

def display_items(self):
    print("\n--- Inventory ---")
    if not self.items:
        print("Inventory is empty!")
    else:
        for name, details in self.items.items():
            print(f"Item: {name}, Price: {details['price']}, Quantity: {details['quantity']}")

def main():
    inv = Inventory()

    while True:
        print("\n1. Add Item")
        print("2. Remove Item")
        print("3. Update Item")
        print("4. Display Items")
        print("5. Exit")

        choice = int(input("Enter choice: "))

        if choice == 1:
            name = input("Enter item name: ")
            price = float(input("Enter price: "))
            qty = int(input("Enter quantity: "))
            inv.add_item(name, price, qty)

        elif choice == 2:
            name = input("Enter item name to remove: ")
            inv.remove_item(name)

        elif choice == 3:
            name = input("Enter item name to update: ")
            price = float(input("Enter new price: "))
            qty = int(input("Enter new quantity: "))
            inv.update_item(name, price, qty)

        elif choice == 4:
            inv.display_items()

        elif choice == 5:
            print("Exiting...")
            break

        else:
            print("Invalid choice!")

main()

```

```

1. Add Item
2. Remove Item
3. Update Item
4. Display Items
5. Exit
Item added successfully!

```

```

1. Add Item
2. Remove Item
3. Update Item
4. Display Items
5. Exit
--- Inventory ---
Item: Pen, Price: 10.0, Quantity: 5

```

```

1. Add Item
2. Remove Item
3. Update Item
4. Display Items
5. Exit
Exiting...

```

07) Create a Class with instance attributes of your choice.

```

In [15]: class Student:

    def __init__(self, name, age, course):
        self.name = name
        self.age = age
        self.course = course

    def display(self):
        print("Name:", self.name)
        print("Age:", self.age)
        print("Course:", self.course)

def main():
    s1 = Student("Bhavik", 18, "Computer Engineering")
    s1.display()

main()

```

```

Name: Bhavik
Age: 18
Course: Computer Engineering

```

08) Create one class student_kit

Within the student_kit class create one class attribute principal name (Mr ABC)

Create one attendance method and take input as number of days.

While creating student take input their name .

Create one certificate for each student by taking input of number of days present in class.

```
In [16]: class student_kit:

    principal_name = "Mr ABC"

    def __init__(self, name):
        self.name = name

    def attendance(self):
        self.days = int(input("Enter number of days present: "))

    def certificate(self):
        print("\n--- Certificate ---")
        print("Student Name:", self.name)
        print("Days Present:", self.days)
        print("Principal:", student_kit.principal_name)

def main():
    n = int(input("Enter number of students: "))

    for i in range(n):
        print("\nEnter details for Student", i + 1)

        name = input("Enter student name: ")
        s = student_kit(name)

        s.attendance()
        s.certificate()

main()
```

```
Enter details for Student 1
--- Certificate ---
Student Name: Bhavik
Days Present: 25
Principal: Mr ABC
```

```
Enter details for Student 2
--- Certificate ---
Student Name: Sunil
Days Present: 30
Principal: Mr ABC
```

09) Define Time class with hour and minute as data member. Also define addition method to add two time objects.

```
In [18]: class Time:

    def __init__(self, hour, minute):
        self.hour = hour
        self.minute = minute

    def add_time(self, t2):
        total_minutes = self.minute + t2.minute
        total_hours = self.hour + t2.hour

        if total_minutes >= 60:
            total_hours += total_minutes // 60
```

```
        total_minutes = total_minutes % 60

    return Time(total_hours, total_minutes)

def display(self):
    print(f"{self.hour} hour(s) {self.minute} minute(s)")

def main():
    # First time
    h1 = int(input("Enter hour for Time1: "))
    m1 = int(input("Enter minute for Time1: "))
    t1 = Time(h1, m1)

    # Second time
    h2 = int(input("Enter hour for Time2: "))
    m2 = int(input("Enter minute for Time2: "))
    t2 = Time(h2, m2)

    result = t1.add_time(t2)

    print("\nResultant Time:")
    result.display()

main()
```

Resultant Time:

4 hour(s) 20 minute(s)